

Esercizio — Unix Socket, FIFO e Segnali

Obiettivo

Viene fornito il file `server.c` che implementa un processo server. Lo scopo dell'esercizio è implementare il programma client `client.c` che soddisfi i seguenti requisiti.

Descrizione del protocollo

Il server (già fornito):

1. crea una Unix socket associata al file:

`/tmp/server-socket`

2. crea una FIFO chiamata:

`/tmp/client-fifo`

3. attende la connessione di *due* client (sarà quindi necessario eseguire il client due volte) mediante la socket
 4. utilizzando questa connessione:
 - riceve da ciascun client il relativo PID
 - risponde a ciascun client inviando una struttura dati contenente:
 - un numero identificativo (1 per il primo client connesso, 2 per il secondo client connesso)
 - il nome della FIFO
-

STEP 1 — Connessione al server

Implementare un client che:

1. crea una Unix socket (`AF_UNIX`)
2. si connette al server su:

`/tmp/server-socket`

3. invia al server il proprio PID
4. riceve dal server una struttura contenente
 - un numero (1 o 2)
 - il nome di una FIFO (che sarà utilizzata nello step successivo)
5. li stampa sullo standard output

Vincoli

Il client deve comunicare con il server solo mediante socket aperta dal server.

Il programma deve avere nome client1.c

STEP 2 — Comunicazione tramite FIFO

Caso 1 — Primo client (`client_number == 1`)

Il client deve:

1. aprire la FIFO in scrittura
2. inviare sulla FIFO il proprio PID

Caso 2 — Secondo client (`client_number == 2`)

Il client deve:

1. aprire la FIFO in lettura
2. leggere dalla FIFO il PID inviato dal primo client
3. stamparlo su standard output

Vincoli

Le due istanze del programma client devono comunicare tra loro esclusivamente mediante la FIFO creata dal server.

Il programma deve avere nome client2.c

STEP 3 — Gestione di SIGUSR1

Caso 1 — Primo client (`client_number == 1`)

Il client deve:

1. installare un handler per SIGUSR1
2. dopo aver inviato il proprio pid sulla FIFO, mettersi in pausa in attesa di segnali
3. quando riceve SIGUSR1, stampare:

`sigusr1 ricevuto!`

e terminare in modo ordinato

Caso 2 — Secondo client (`client_number == 2`)

Il client deve:

1. leggere dalla FIFO il PID del primo client
2. inviare a quel PID il segnale:

`SIGUSR1`

3. terminare in modo ordinato

Vincoli

Il programma client deve avere nome `client3.c`
